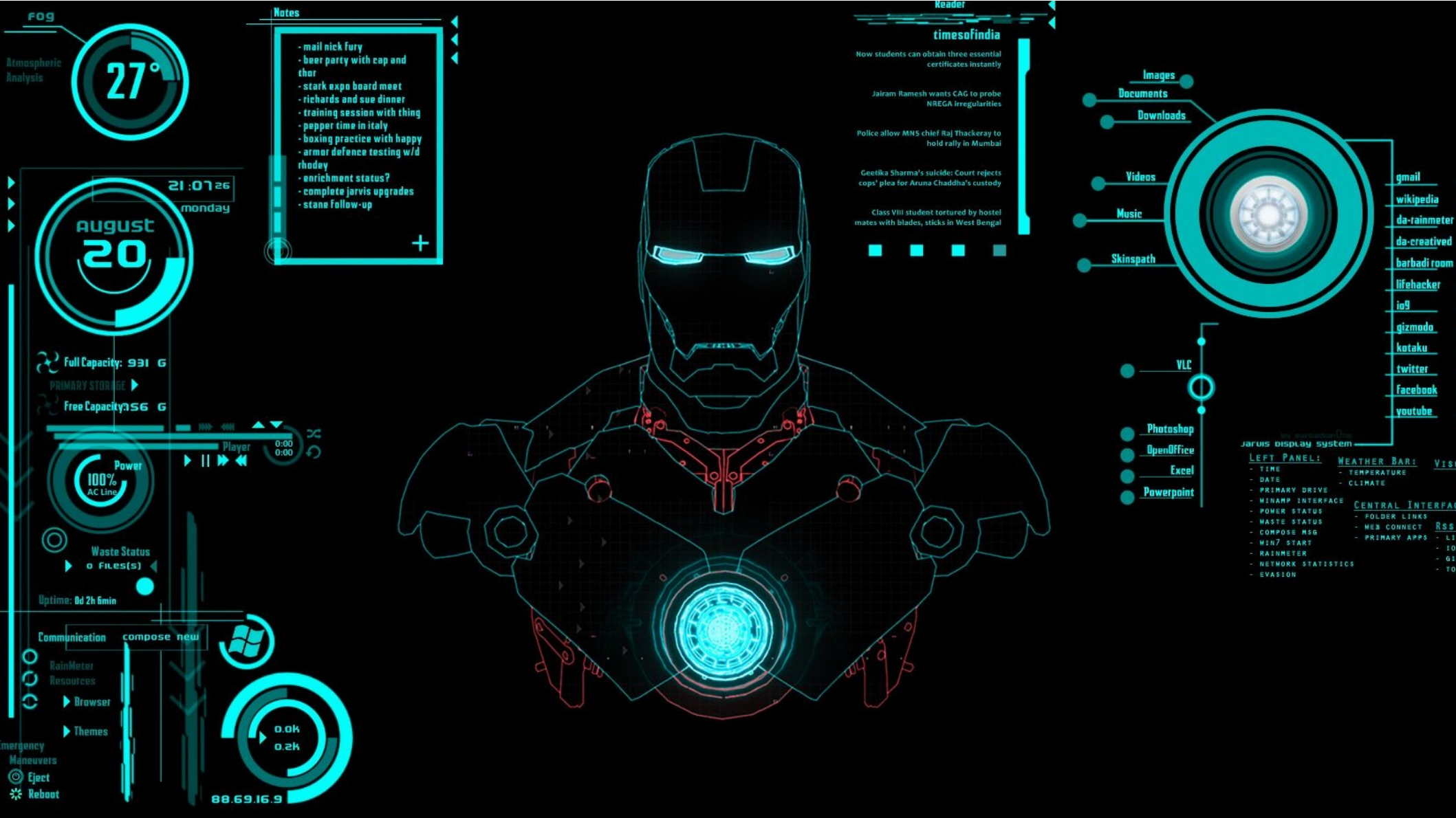




16.Cores e Iluminação Parte 2

Prof. Alexandre Krohn

Roteiro

- Vetores Normais e Transformações
- Iluminação Especular
- Gourad vs. Phong
- Materiais

Vetores Normais

- Na aula anterior, passamos o vetor normal diretamente **do shader de vértices para o shader de fragmentos** do objeto.
- No entanto, os cálculos no shader de fragmentos são todos feitos no espaço mundial
- **Não deveríamos então transformar os vetores normais em coordenadas do espaço mundial também?**

Vetores Normais

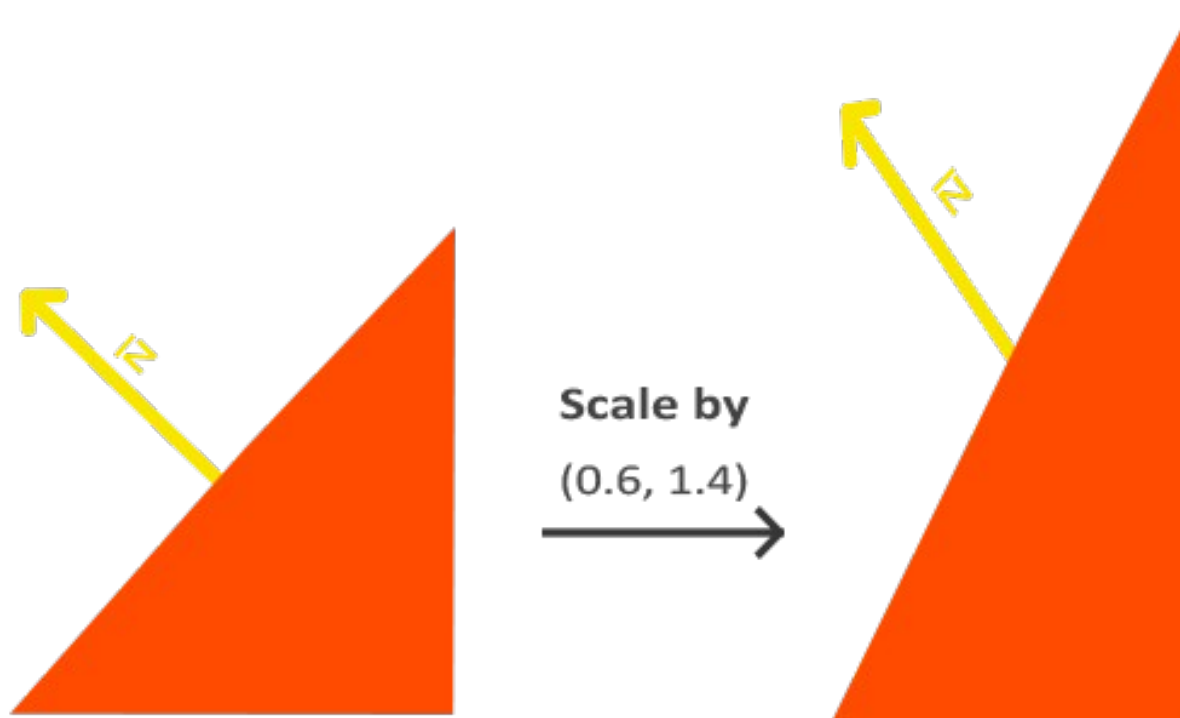
- **Sim**, deveríamos
- Mas dois problemas surgem dessa abordagem:

Problemas

- **1)** Vetores normais são apenas vetores de direção e **não representam** uma posição específica no espaço.
- Isso significa que as **traduções não apresentam nenhum efeito** nos vetores normais.

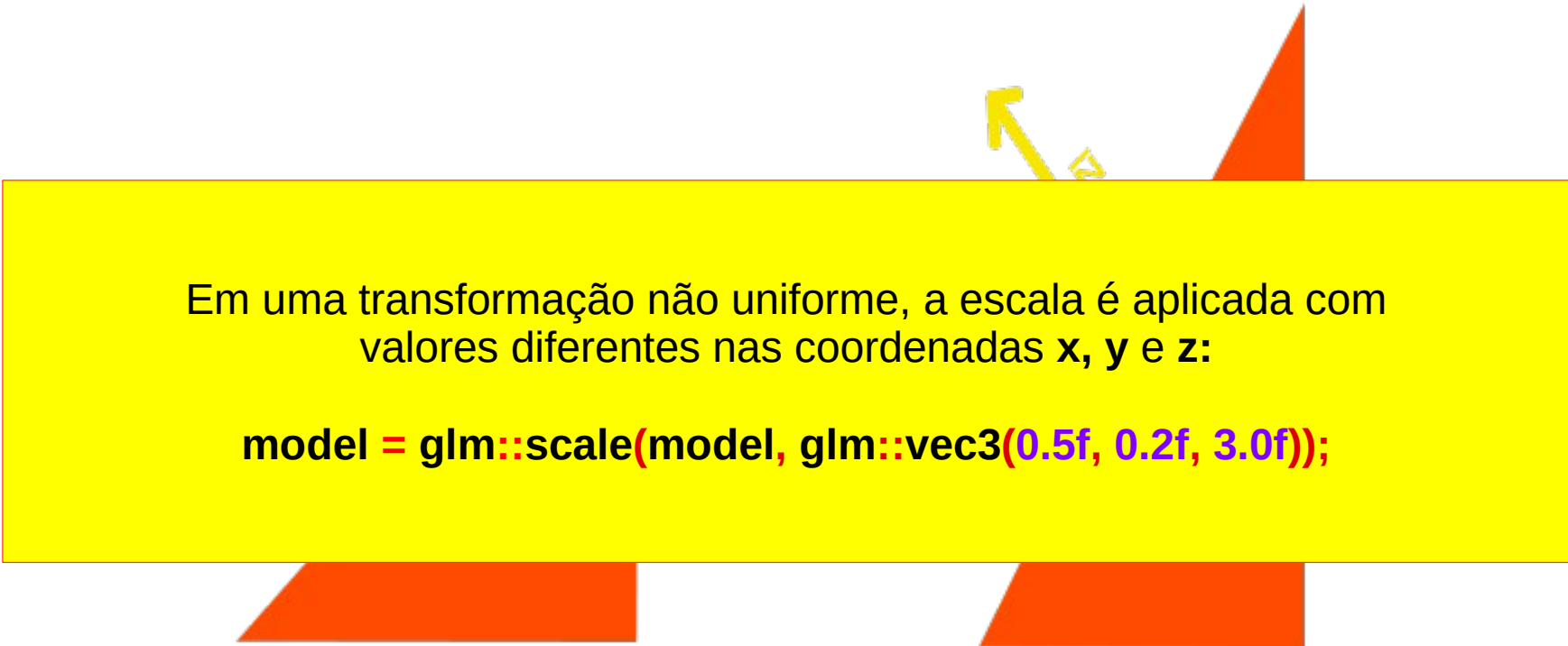
Problemas

- **2)** Se a matriz do modelo executasse uma escala não uniforme, os vértices seriam alterados de forma que o vetor normal não seria mais perpendicular à superfície.



Problemas

- **2)** Se a matriz do modelo executasse uma escala não uniforme, os vértices seriam alterados de forma que o vetor normal não seria mais perpendicular à superfície.



Em uma transformação não uniforme, a escala é aplicada com valores diferentes nas coordenadas **x**, **y** e **z**:

```
model = glm::scale(model, glm::vec3(0.5f, 0.2f, 3.0f));
```

Problemas

- Sempre que aplicamos uma **escala não uniforme** (nota: uma escala uniforme apenas muda a magnitude da normal, não sua direção, que é facilmente corrigida normalizando-a) os vetores normais **não são mais perpendiculares** à superfície correspondente, o que distorce a iluminação.



Solução

- O truque para corrigir esse comportamento é **usar uma matriz de modelo diferente**, adaptada especificamente para vetores normais.
- Essa matriz é chamada de matriz normal e é **“a transposição do inverso da parte superior esquerda 3x3 da matriz do modelo”**

Solução

- No shader de vértices, podemos gerar a matriz normal usando as funções inversa e transposta, que funcionam em qualquer tipo de matriz.
- Observe que lançamos a matriz em uma matriz 3x3 para garantir que ela perca suas propriedades de translação e que possa se multiplicar com o vetor normal vec3:

Normal = mat3(transpose(inverse(model))) * aNormal;

Solução : Shaders

- Se você estiver fazendo uma escala não uniforme é essencial que você multiplique seus vetores normais pela matriz normal.

Shader de Vértices

```
#version 330 core
```

```
layout (location = 0) in vec3 aPos;
```

```
layout (location = 1) in vec3 aNormal;
```

```
out vec3 FragPos;
```

```
out vec3 Normal;
```

```
out vec3 LightPos;
```

```
uniform vec3 lightPos;
```

```
// we now define the uniform in the vertex shader and pass the 'view space' lightpos to  
// the fragment shader. lightPos is currently in world space.
```

```
uniform mat4 model;
```

```
uniform mat4 view;
```

```
uniform mat4 projection;
```

```
void main()
```

```
{
```

```
    gl_Position = projection * view * model * vec4(aPos, 1.0);
```

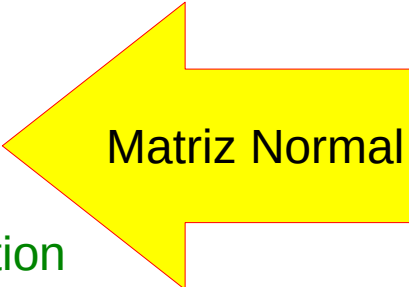
```
    FragPos = vec3(view * model * vec4(aPos, 1.0));
```

```
    Normal = mat3(transpose(inverse(view * model))) * aNormal;
```

```
    LightPos = vec3(view * vec4(lightPos, 1.0));
```

```
    // Transform world-space light position to view-space light position
```

```
}
```



Matriz Normal

Shader de Fragmentos

```
#version 330 core
out vec4 FragColor;
in vec3 FragPos;
in vec3 Normal;
in vec3 LightPos; // extra in variable, since we need the light position in view space we calculate
// this in the vertex shader

uniform vec3 lightColor;
uniform vec3 objectColor;
uniform float specularStrength;

void main()
{
    // ambient
    float ambientStrength = 0.1;
    vec3 ambient = ambientStrength * lightColor;

    // diffuse
    vec3 norm = normalize(Normal);
    vec3 lightDir = normalize(LightPos - FragPos);
    float diff = max(dot(norm, lightDir), 0.0);
    vec3 diffuse = diff * lightColor;

    // specular
    vec3 viewDir = normalize(-FragPos); // the viewer is always at (0,0,0) in view-space, so
    // viewDir is (0,0,0) - Position => -Position
    vec3 reflectDir = reflect(-lightDir, norm);
    float spec = pow(max(dot(viewDir, reflectDir), 0.0), 32);
    vec3 specular = specularStrength * spec * lightColor;
    vec3 result = (ambient + diffuse + specular) * objectColor;
    FragColor = vec4(result, 1.0);
}
```

Por fim

- A inversão de matrizes é uma operação cara para os shaders, portanto, sempre que possível, tente evitar fazer operações inversas, uma vez que elas devem ser feitas em cada vértice de sua cena.
- Para fins de aprendizado, tudo bem, mas para um aplicativo eficiente, você provavelmente desejará calcular a matriz normal na CPU e enviá-la aos shaders por meio de um ***uniform*** antes de desenhar (assim como a matriz do modelo).

Roteiro

- Vetores Normais e Transformações
- Iluminação Especular
- Gourad vs. Phong
- Materiais

Iluminação Especular

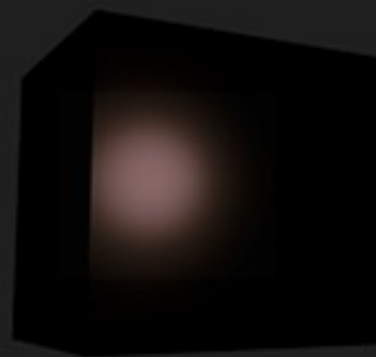
- Já analisamos os dois primeiros blocos de construção do modelo de iluminação **Phong**:
 - Iluminação ambiente;
 - Iluminação difusa e
 - Agora veremos **Iluminação especular**.



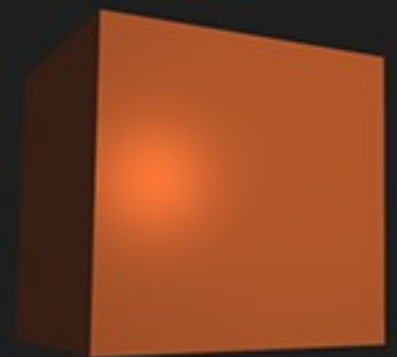
ambient



diffuse



specular

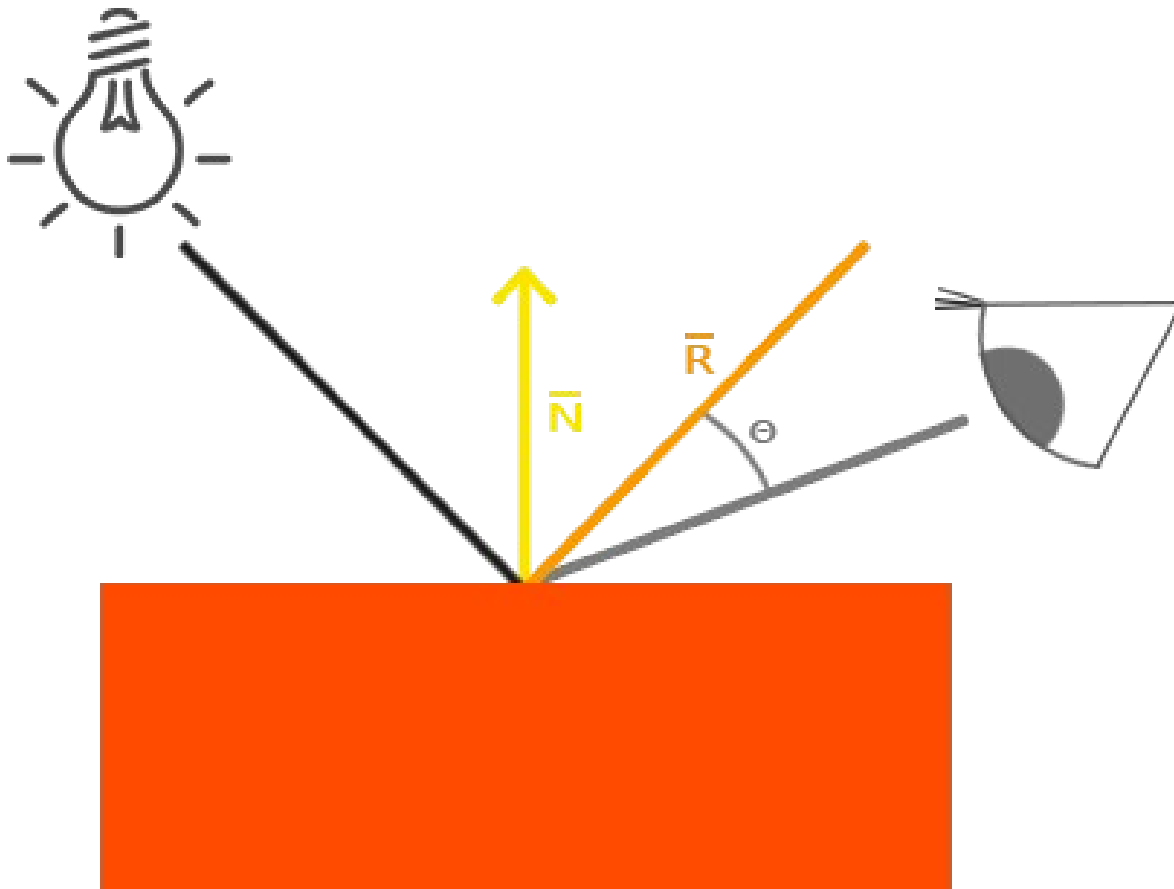


combined (Phong)

Iluminação Especular

- Semelhante à iluminação difusa, a iluminação especular também é baseada no vetor de direção da luz e nos vetores normais do objeto,
- Mas desta vez também **leva em conta o ponto de vista**, ou seja, **de que direção o observador está olhando para o fragmento.**

Iluminação Especular



- O reflexo gerado pela fonte de luz é importante aqui

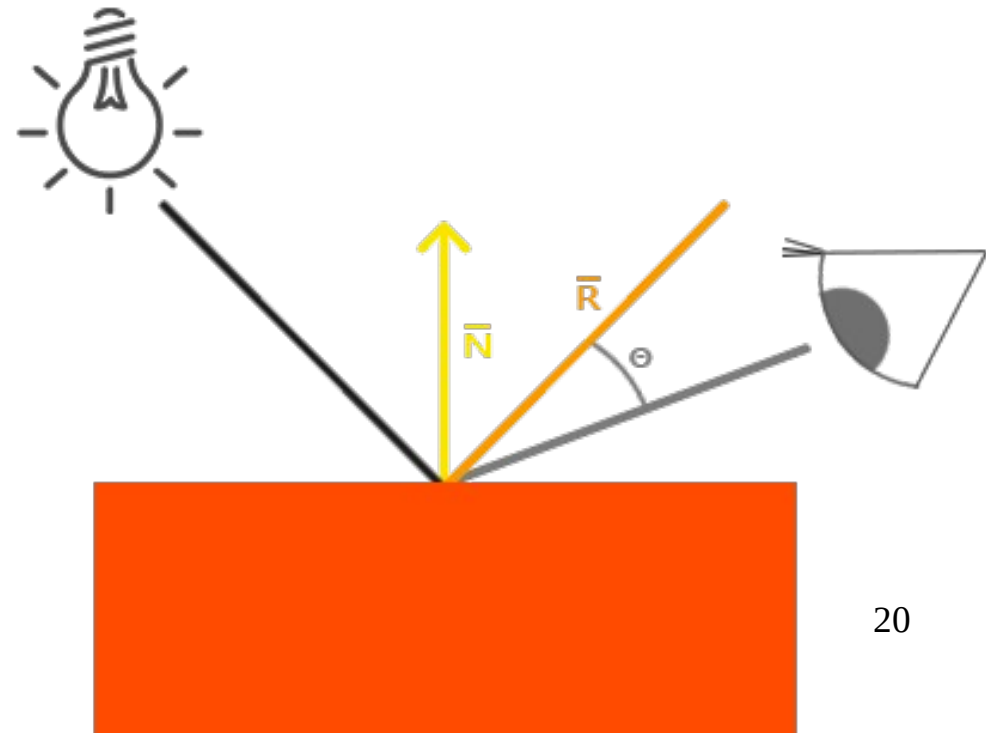
Iluminação Especular

- A iluminação especular é baseada nas propriedades reflexivas das superfícies.
- Se pensarmos na superfície do objeto como um espelho, a iluminação especular é mais forte no ponto onde vemos a fonte de luz refletida na superfície.



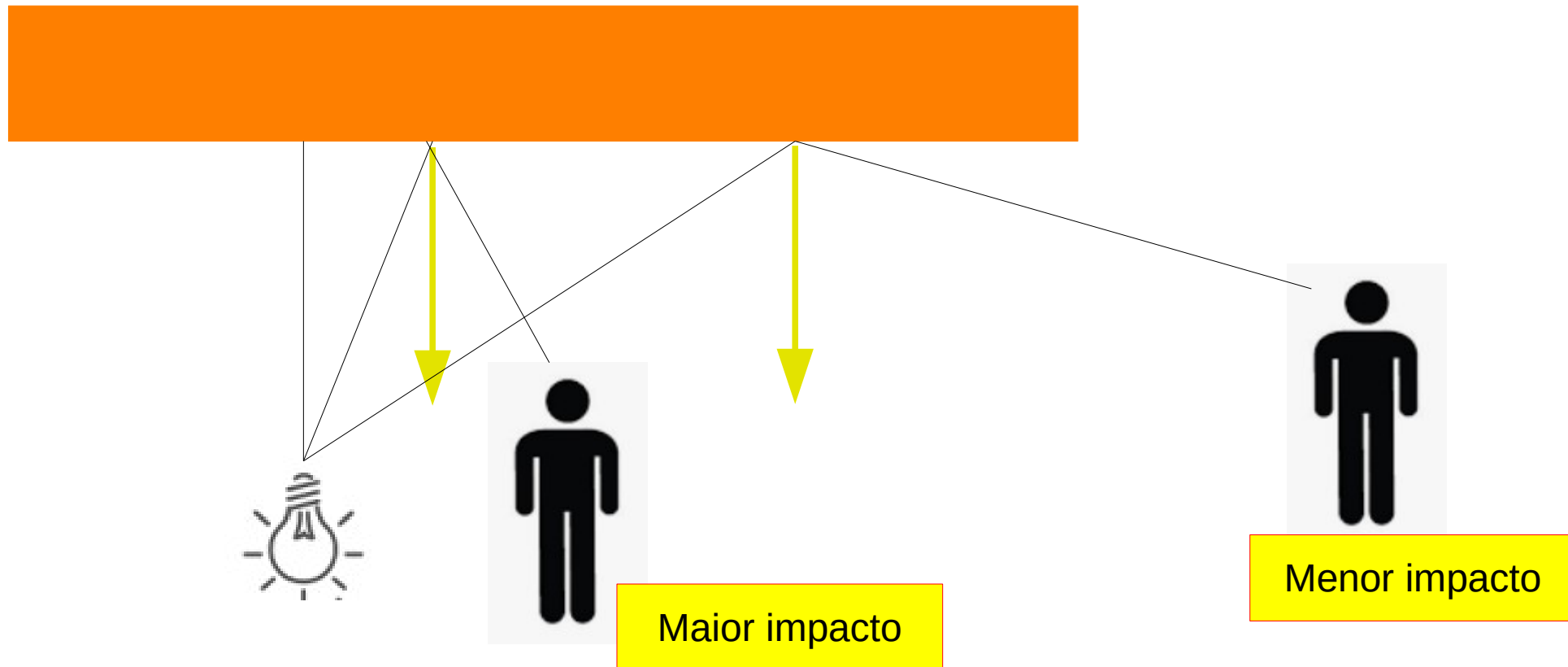
Iluminação Especular

- Calcula-se um vetor de reflexão refletindo a direção da luz em torno do vetor normal.
- Em seguida, calcula-se a distância angular entre este vetor de reflexão e a direção da vista.



Iluminação Especular

- Quanto mais próximo o ângulo entre eles, maior o impacto da luz especular.



Iluminação Especular

- É necessária uma variável extra para calcular a iluminação especular : **O vetor de visualização**.
- Pode-se calcular o vetor de iluminação usando a posição do espaço mundial do observador e a posição do fragmento.
- Então calcula-se a intensidade do reflexo especular, multiplica-se pela cor da luz e adiciona-se o resultado aos componentes ambiente e difuso.

Cálculo no shader

- Para obter as coordenadas do espaço mundial do visualizador, simplesmente pega-se o vetor de posição do objeto de câmera (que é o observador).
- É adicionado outro ***uniform*** ao shader de fragmentos e passado o vetor de posição da câmera para o shader:

```
// no shader de fragmentos  
uniform vec3 viewPos;
```

```
// no código em C  
lightingShader.setVec3("viewPos", camera.Position);
```

Intensidade Especular

- Define-se um valor de intensidade especular para dar ao realce especular uma cor de brilho médio para que não tenha muito impacto:

```
// no shader de fragmentos  
float specularStrength = 0.5;
```

Quanto maior o valor da intensidade especular, maior o reflexo.

Cálculo da Reflexão

- Calcula-se o vetor de direção da vista e o vetor de reflexão correspondente ao longo do eixo normal:

// no shader de fragmentos

```
vec3 viewDir = normalize(viewPos - FragPos);  
vec3 reflectDir = reflect(-lightDir, norm);
```

Observe que negamos o vetor lightDir. A função de reflexão espera que o primeiro vetor aponte da fonte de luz para a posição do fragmento, mas o vetor lightDir está apontando atualmente para o outro lado: do fragmento para a fonte de luz (isso depende da ordem de subtração anterior quando calculamos o vetor lightDir). Para ter certeza de obter o vetor de reflexão correto, invertemos sua direção, negando primeiro o vetor lightDir. O segundo argumento espera um vetor normal, então fornecemos o vetor normal normalizado.

Cálculo da Reflexão

- Então, o que resta fazer é calcular o componente especular. Isso é feito com a seguinte fórmula:

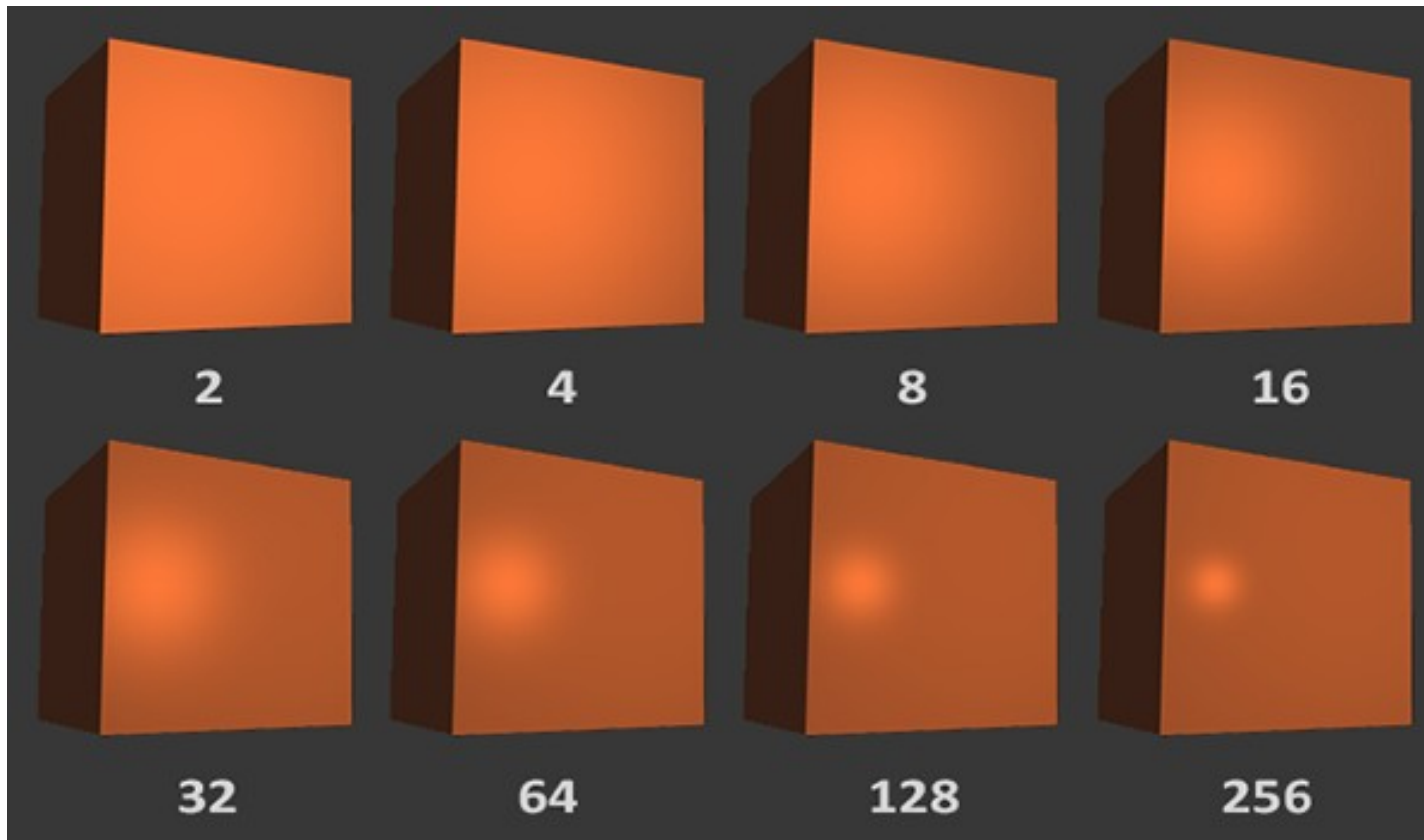
// no shader de fragmentos

```
float spec = pow(max(dot(viewDir, reflectDir), 0.0), 32);  
vec3 specular = specularStrength * spec * lightColor;
```

Primeiro calculamos o produto escalar entre a direção da visualização e a direção do reflexo (e nos certificamos de que não é negativo) e, em seguida, aumentamos para a **potência de 32**.

Potência de 32?

- O valor 32 é o valor de brilho do realce. Quanto mais alto o valor de brilho de um objeto, mais ele reflete corretamente a luz em vez de espalhá-la ao redor e, portanto, menor será o realce.



Cálculo da Iluminação

- A única coisa que resta a fazer é adicionar o componente especular aos componentes ambiente e difuso e multiplicar o resultado combinado pela cor do objeto:

// no shader de fragmentos

```
vec3 result = (ambient + diffuse + specular) * objectColor;  
FragColor = vec4(result, 1.0);
```

Cálculo da Iluminação

- Nos primeiros dias dos shaders de iluminação, os desenvolvedores costumavam implementar o modelo de iluminação Phong no **shader de vértices**.
- A vantagem de fazer iluminação no shader de vértices é que é muito mais eficiente, pois geralmente há muito menos vértices em comparação com fragmentos, **portanto, os (caros) cálculos de iluminação são feitos com menos frequência**.

Cálculo da Iluminação

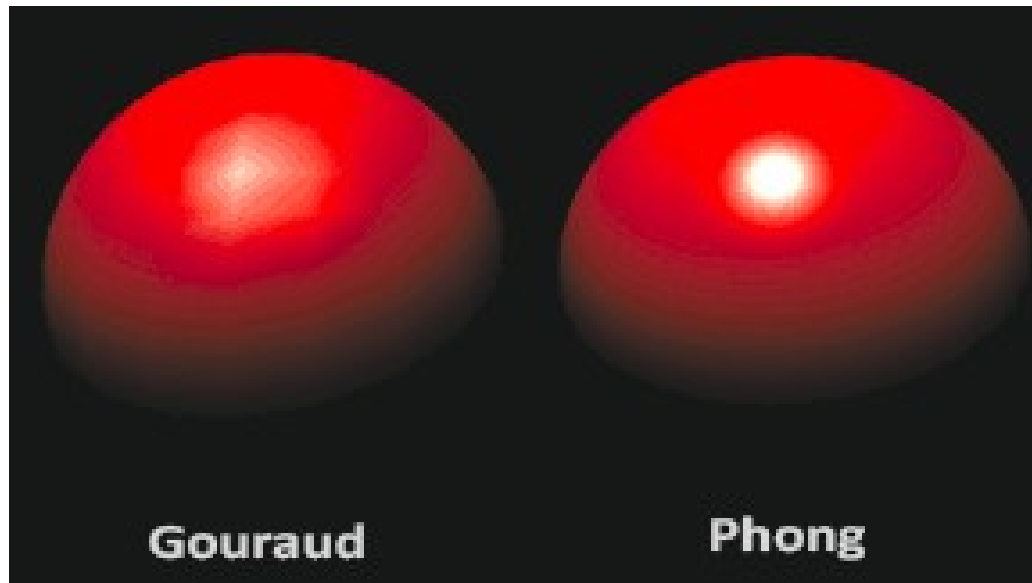
- No entanto, o valor de cor resultante no sombreador de vértice é a cor de iluminação resultante apenas desse vértice e os valores de cor dos fragmentos circundantes são, então, o resultado de cores de iluminação interpoladas.
- O resultado foi que a iluminação **não era muito realista**, a menos que grandes quantidades de vértices fossem usadas

Roteiro

- Vetores Normais e Transformações
- Iluminação Especular
- Gourad vs. Phong
- Materiais

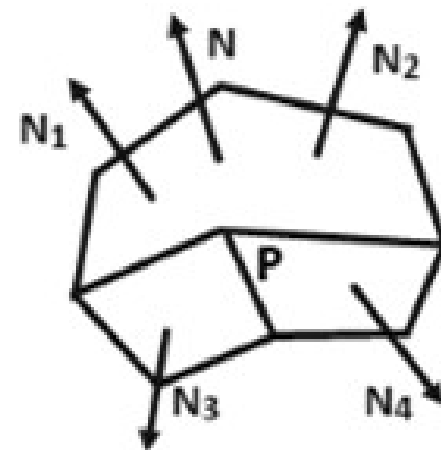
Gourad vs. Phong

- Quando o modelo de iluminação Phong é implementado no shader de vértices, ele é chamado de **Gouraud Shading** em vez de Phong Shading.
- Observe que, devido à interpolação, a iluminação parece um pouco apagada. O Phong Shading oferece resultados de iluminação muito mais suaves.



Gouraud Shading

- Cada superfície do polígono é renderizada com Gouraud Shading executando os seguintes cálculos:
 - Determina-se a média do vetor normal de cada vértice do polígono.
 - Aplica-se um modelo de iluminação a cada vértice para determinar a intensidade do vértice.
 - Realiza-se a interpolação linear das intensidades dos vértices sobre a superfície do polígono.
- Em cada vértice do polígono, obtemos um vetor normal calculando a média das normais da superfície de todos os polígonos que partem desse vértice



Roteiro

- Vetores Normais e Transformações
- Iluminação Especular
- Gourad vs. Phong
- Materiais

Materiais

- No mundo real, **cada objeto tem uma reação diferente à luz.**
 - Os objetos de aço costumam ser mais brilhantes do que um vaso de barro, por exemplo, e um recipiente de madeira não reage à luz da mesma forma que um recipiente de aço.
 - Alguns objetos refletem a luz sem muita dispersão, resultando em pequenos realces especulares, e outros se espalham muito, dando ao realce um raio maior.

Materiais

- Se quisermos simular vários tipos de objetos em OpenGL, temos que definir as **propriedades do material específicas para cada superfície.**

Materiais

- Até aqui, definimos um objeto e uma cor de luz para definir a saída visual do objeto, combinada com um componente de intensidade especular e ambiente.
- Ao descrever uma superfície, podemos definir uma cor de material para cada um dos 3 componentes de iluminação: iluminação ambiente, difusa e especular.
- Ao especificar uma cor para cada um dos componentes, temos um **controle refinado** sobre a saída de cor da superfície.
- Agora adicione um componente de brilho a essas 3 cores e teremos todas as propriedades do material de que precisamos:

Materiais : Fragment Shader

- No shader de fragmento, criamos uma estrutura para armazenar as propriedades do material da superfície.
- Primeiro definimos o layout da estrutura e, em seguida, simplesmente declaramos uma variável ***uniform*** com a estrutura recém-criada como seu tipo.

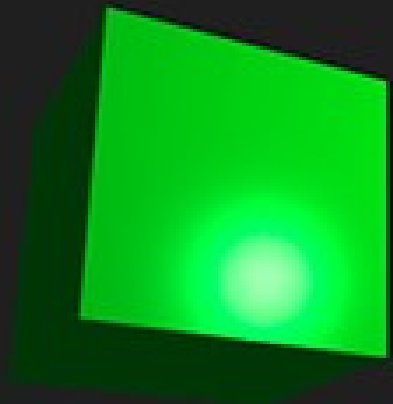
```
#version 330 core
struct Material {
    vec3 ambient;
    vec3 diffuse;
    vec3 specular;
    float shininess;
};
```

```
uniform Material material;
```

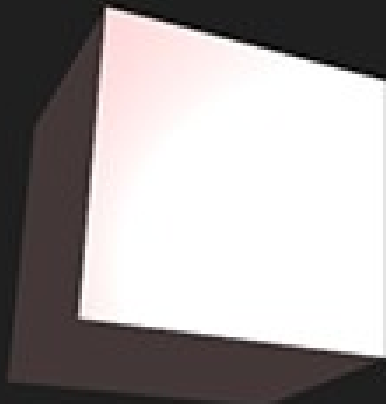
Materiais : Cores

- Define-se um vetor de cor para cada um dos componentes da iluminação Phong.
 - O vetor de **material ambiente** define a cor que a superfície reflete sob a iluminação ambiente; geralmente é igual à cor da superfície.
 - O vetor de **material difuso** define a cor da superfície sob iluminação difusa. A cor difusa é (assim como a iluminação ambiente) definida para a cor da superfície desejada.
 - O vetor de **material especular** define a cor do realce especular na superfície (ou possivelmente reflete uma cor específica da superfície).
 - Por último, o **brilho** impacta a dispersão / raio do realce especular.

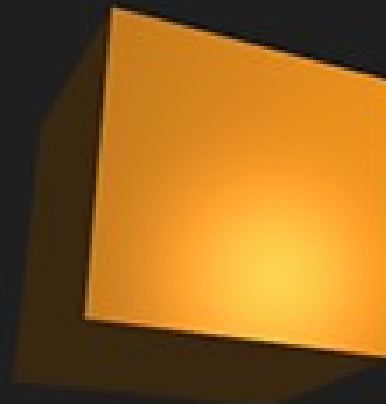
Materials : Cores



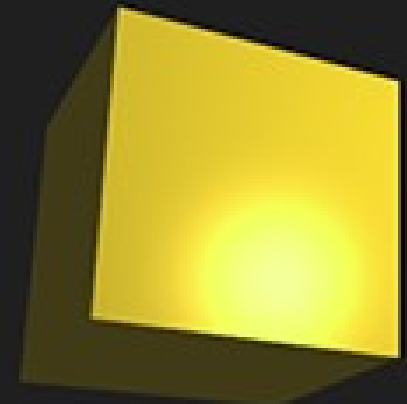
Emerald



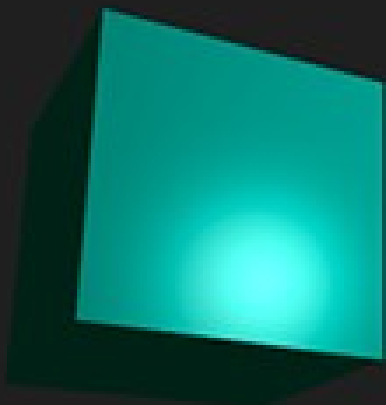
Pearl



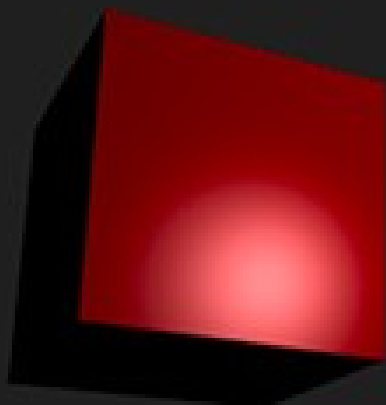
Bronze



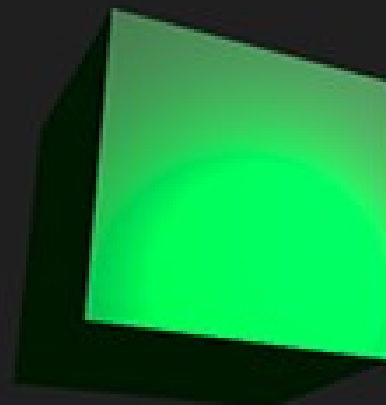
Gold



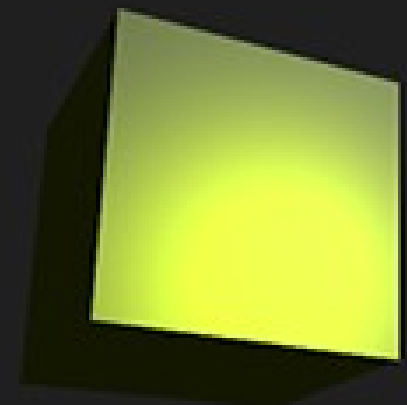
Cyan Plastic



Red Plastic



Green Rubber



Yellow Rubber

Materiais : Cores

- Com esses 4 componentes que definem o material de um objeto, podemos simular muitos materiais do mundo real.
- Há uma tabela no site devernay.free.fr que mostra uma lista de propriedades de materiais que simulam materiais reais encontrados no mundo exterior.
- O slide seguinte mostra o efeito que vários desses valores materiais do mundo real têm em nosso cubo:

Tabela de materiais

Multiplica-se o brilho por 128 em OpenGL

Name	Ambient			Diffuse			Specular			Shininess
emerald	0.0215	0.1745	0.0215	0.07568	0.61424	0.07568	0.633	0.727811	0.633	0.6
jade	0.135	0.2225	0.1575	0.54	0.89	0.63	0.316228	0.316228	0.316228	0.1
obsidian	0.05375	0.05	0.06625	0.18275	0.17	0.22525	0.332741	0.328634	0.346435	0.3
pearl	0.25	0.20725	0.20725	1	0.829	0.829	0.296648	0.296648	0.296648	0.088
ruby	0.1745	0.01175	0.01175	0.61424	0.04136	0.04136	0.727811	0.626959	0.626959	0.6
turquoise	0.1	0.18725	0.1745	0.396	0.74151	0.69102	0.297254	0.30829	0.306678	0.1
brass	0.329412	0.223529	0.027451	0.780392	0.568627	0.113725	0.992157	0.941176	0.807843	0.21794872
bronze	0.2125	0.1275	0.054	0.714	0.4284	0.18144	0.393548	0.271906	0.166721	0.2
chrome	0.25	0.25	0.25	0.4	0.4	0.4	0.774597	0.774597	0.774597	0.6
copper	0.19125	0.0735	0.0225	0.7038	0.27048	0.0828	0.256777	0.137622	0.086014	0.1
gold	0.24725	0.1995	0.0745	0.75164	0.60648	0.22648	0.628281	0.555802	0.366065	0.4
silver	0.19225	0.19225	0.19225	0.50754	0.50754	0.50754	0.508273	0.508273	0.508273	0.4
black plastic	0.0	0.0	0.0	0.01	0.01	0.01	0.50	0.50	0.50	.25
cyan plastic	0.0	0.1	0.06	0.0	0.50980392	0.50980392	0.50196078	0.50196078	0.50196078	.25
green plastic	0.0	0.0	0.0	0.1	0.35	0.1	0.45	0.55	0.45	.25
red plastic	0.0	0.0	0.0	0.5	0.0	0.0	0.7	0.6	0.6	.25
white plastic	0.0	0.0	0.0	0.55	0.55	0.55	0.70	0.70	0.70	.25
yellow plastic	0.0	0.0	0.0	0.5	0.5	0.0	0.60	0.60	0.50	.25
black rubber	0.02	0.02	0.02	0.01	0.01	0.01	0.4	0.4	0.4	.078125
cyan rubber	0.0	0.05	0.05	0.4	0.5	0.5	0.04	0.7	0.7	.078125
green rubber	0.0	0.05	0.0	0.4	0.5	0.4	0.04	0.7	0.04	.078125
red rubber	0.05	0.0	0.0	0.5	0.4	0.4	0.7	0.04	0.04	.078125
white rubber	0.05	0.05	0.05	0.5	0.5	0.5	0.7	0.7	0.7	.078125
yellow rubber	0.05	0.05	0.0	0.5	0.5	0.4	0.7	0.7	0.04	.078125

Implementação do Shader

- Foi definida uma estrutura Material no shader de fragmentos, e uma variável ***uniform***
- Deve-se agora alterar os cálculos de iluminação para atender às novas propriedades do material.
- Uma vez que todas as variáveis de material são armazenadas em uma estrutura, podemos acessá-las a partir da ***uniform*** de material:

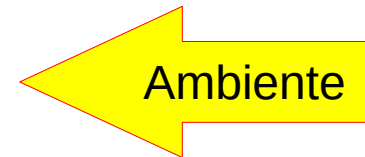
Fragment Shader

```
void main()
{
    // ambient
    vec3 ambient = lightColor * material.ambient;

    // diffuse
    vec3 norm = normalize(Normal);
    vec3 lightDir = normalize(lightPos - FragPos);
    float diff = max(dot(norm, lightDir), 0.0);
    vec3 diffuse = lightColor * (diff * material.diffuse);

    // specular
    vec3 viewDir = normalize(viewPos - FragPos);
    vec3 reflectDir = reflect(-lightDir, norm);
    float spec = pow(max(dot(viewDir, reflectDir), 0.0), material.shininess);
    vec3 specular = lightColor * (spec * material.specular);

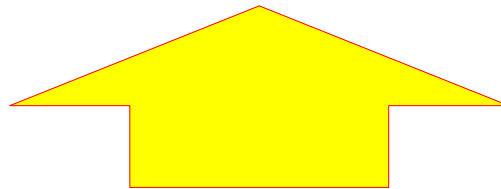
    vec3 result = ambient + diffuse + specular;
    FragColor = vec4(result, 1.0);
}
```



Uniform Struct

- E como se passam os valores para uma Struct através de *uniforms*?
- Usando-se “prefixos”!

```
lightingShader.setVec3("material.ambient", 1.0f, 0.5f, 0.31f);  
lightingShader.setVec3("material.diffuse", 1.0f, 0.5f, 0.31f);  
lightingShader.setVec3("material.specular", 0.5f, 0.5f, 0.5f);  
lightingShader.setFloat("material.shininess", 32.0f);
```

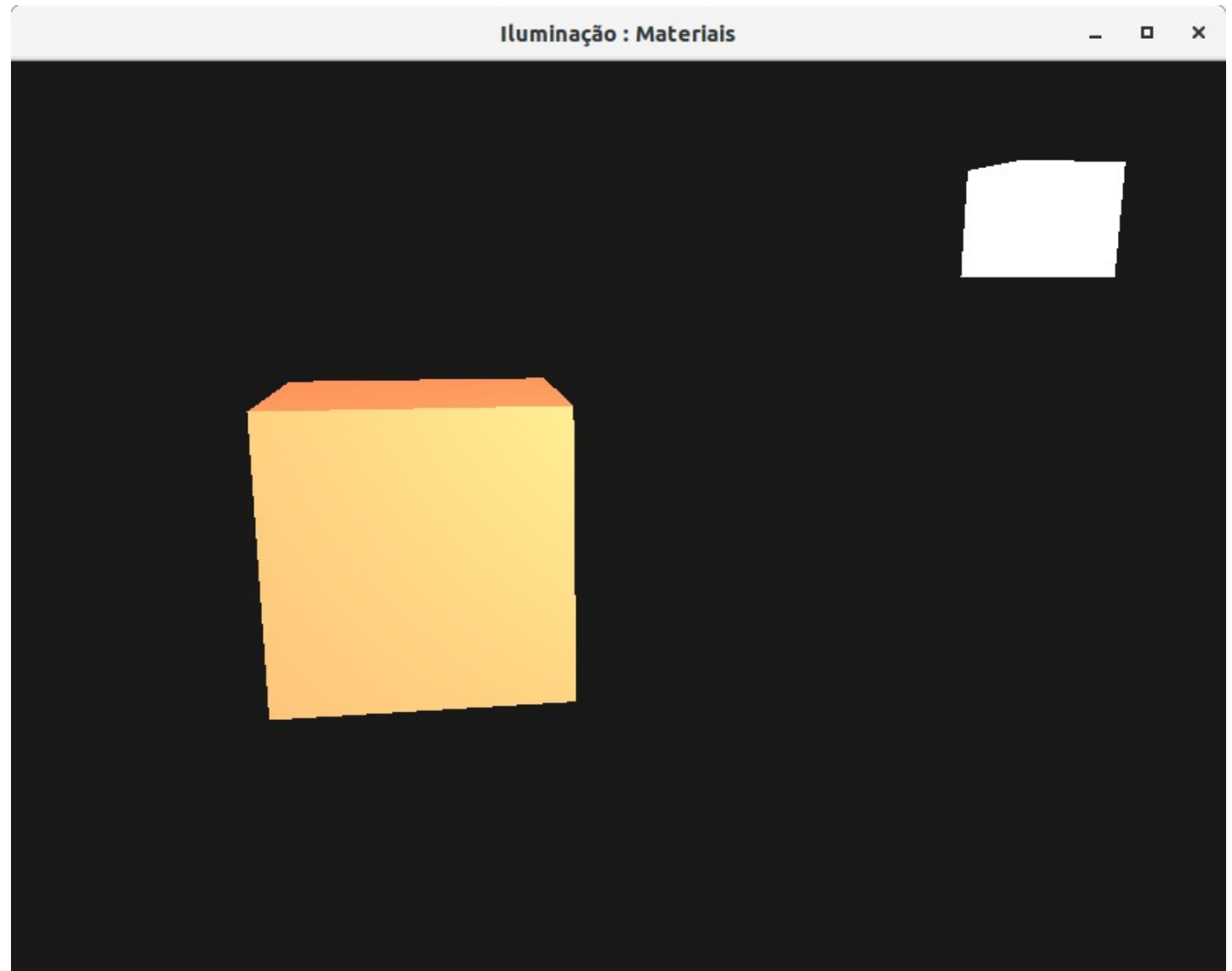


Configurando Materiais

- Definimos o componente **ambiente e difuso** para a **cor que gostaríamos que o objeto tivesse** e definimos o componente **especular** do objeto para uma **cor de brilho médio**; não queremos que o componente especular seja muito forte.
- Também mantemos o brilho em 32.
- Agora podemos influenciar facilmente o material do objeto a partir do aplicativo.

Configurando Materiais

- Agora podemos influenciar facilmente o material do objeto a partir do aplicativo.
- A execução do programa gera algo assim:



Configurando Materiais

- Agora podemos influenciar facilmente o material do objeto a partir do aplicativo.
- A execução do programa gera algo assim:



Propriedades da Luz

- A razão para o objeto ser muito brilhante é que as cores ambientais, difusas e especulares **são refletidas com força total** de qualquer fonte de luz.
- As fontes de luz também têm intensidades diferentes para seus componentes ambiente, difuso e especular, respectivamente.

Propriedades da Luz

- É necessário reduzir intensidade da luz ambiente, assim como da luz especular.
- Para tanto, cria-se outra estrutura no shader de fragmentos, para que possamos configurar os valores da luz

Propriedades da Luz

- Estrutura para luz

```
struct Light {  
    vec3 position;  
    vec3 ambient;  
    vec3 diffuse;  
    vec3 specular;  
};  
  
uniform Light light;
```

Propriedades da Luz

- Uma fonte de luz tem uma intensidade diferente para seus componentes ambientais, difusos e especulares.
 - A luz **ambiente** geralmente é definida para uma intensidade baixa porque não queremos que a cor ambiente seja muito dominante.
 - O componente **difuso** de uma fonte de luz geralmente é definido com a cor exata que gostaríamos que a luz tivesse; frequentemente uma cor branca brilhante.
 - O componente **especular** é geralmente mantido em vec3 (1.0) brilhando com intensidade total.
 - Observe que também foi **adicionado o vetor de posição da luz** à estrutura.

Propriedades da Luz

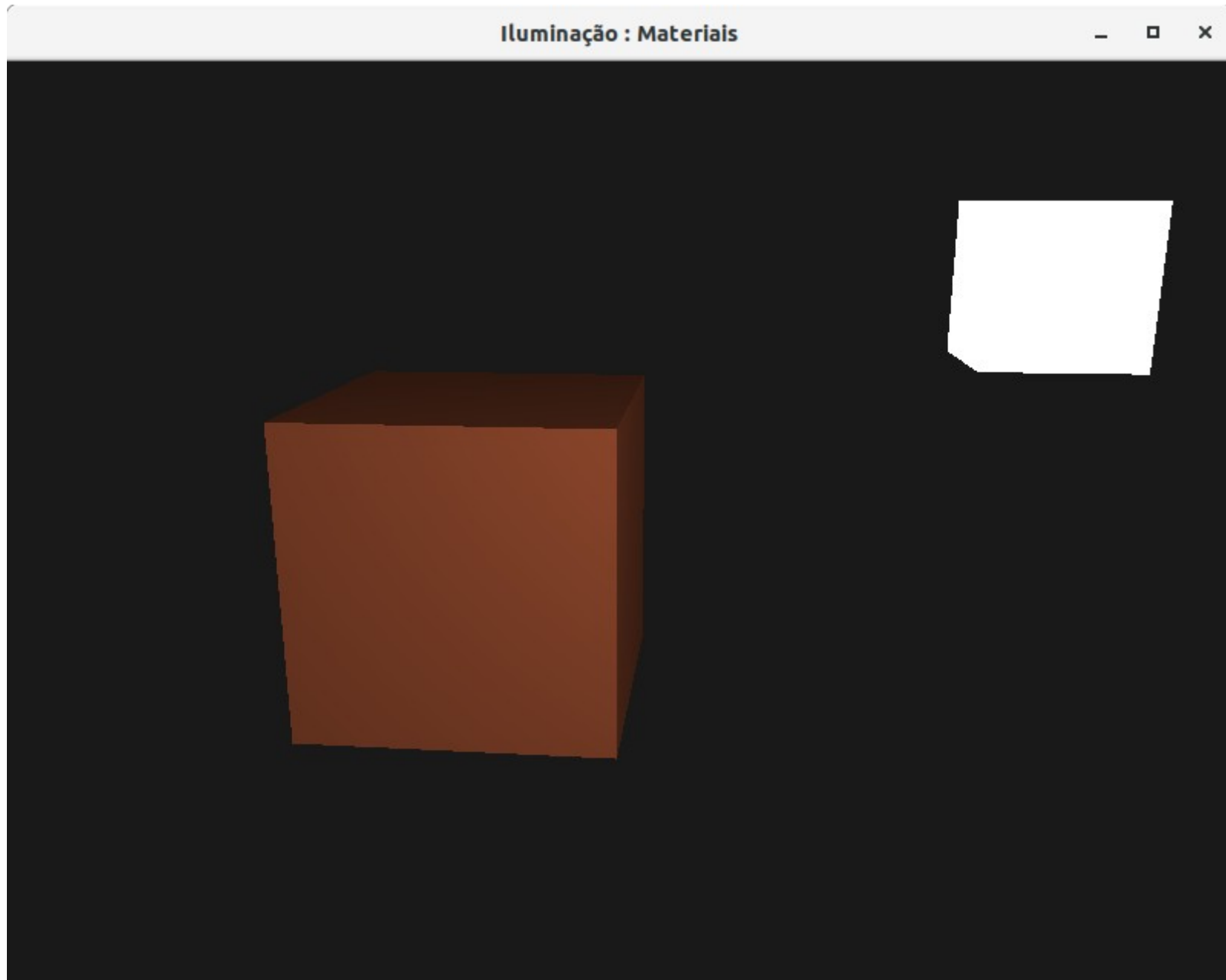
- É necessário atualizar o shader de fragmentos

```
vec3 ambient = light.ambient * material.ambient;  
vec3 diffuse = light.diffuse * (diff * material.diffuse);  
vec3 specular = light.specular * (spec * material.specular);
```

- E definir as cores no aplicativo

```
lightingShader.setVec3("light.ambient", 0.2f, 0.2f, 0.2f);  
// darken diffuse light a bit  
lightingShader.setVec3("light.diffuse", 0.5f, 0.5f, 0.5f);  
lightingShader.setVec3("light.specular", 1.0f, 1.0f, 1.0f);
```

Resultado



Exemplos Fornecidos

- No projeto **GLFW_Iluminacao_2** pode-se observar os efeitos da iluminação pelo modelo de **Phong**.
- O projeto contém os shaders para exemplificar a iluminação pelos modelos de Phong e Gourad
- Além disso, as letras **O** e **L**, quando pressionadas, alteram a intensidade do reflexo especular

Exemplos Fornecidos

- No projeto **GLFW_Iluminacao_3** contém as lógicas para manipulação independente de cada componente de iluminação, permitindo a simulação de materiais diferentes

Dúvidas?



Atividade 1

- Faça o download e execute em seu computador o projeto **GLFW_Iluminacao_2**.
- Depois, altere a quantidade de reflexo pressionando as teclas **O** e **L** e verifique os resultados.
- Edite o arquivo **phong_lighting.fs**, e altere o valor 32 (linha 29) para **2, 4, 8, 16, 64, 128 e 256**. Para cada alteração, pressione as teclas **O** e **L** novamente e observe os resultados
- Substitua os shaders de **Phong** pelos de **Gourad** fornecidos (Já está no código, basta comentar uma linha e descomentar outra). Observe os resultados

Atividade 2

- Faça o download e execute em seu computador o projeto **GLFW_Iluminacao_3**.
- Depois, localize e descomente as linhas abaixo de :
`// Para a cor ficar se alterando sozinha`
- Não esqueça de comentar a linha abaixo do comentário:
`// Luz Branca`
- Observe os resultados no cubo iluminado